Modeling the Vision Transformer in SystemC

EECE 7368 Project - 2024

Aniket, Baibhav, Luyue, Robbie, Tilak

What is a Vision Transformer (ViT)?

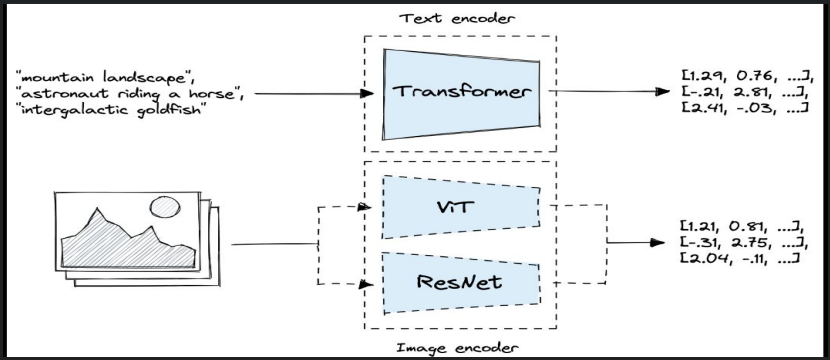
- A neural network architecture for image classification
- Based on the Transformer architecture originally developed for natural language processing (NLP)

ViT → "Bullfrog"

ViT Use Cases

- Healthcare and Medical Diagnosis
- Autonomous Vehicles
- Security and Surveillance
- Large multi modal models (ViT use in CLIP, DALL-E)

Classifying Pneumonia



ViT in OpenAI Clip - Multi modal model

ViT Drawbacks

Deploying the ViT to an embedded application is difficult.

- Compute intensive with many FLOPs and GEMM instructions
  - Very long inference time on small processors
  - High energy consumption
- Memory intensive → requires over 1GB to store the weights
  - Many small SoCs do not have this much memory to spare
  - Smaller caches and register files mean even more time and energy spent on moving weights in and out of cores

Solution: Hardened ViT

Idea: Implement the Vision Transformer processing in hardware

Solution: Hardened ViT

Idea: Implement the Vision Transformer processing in hardware

Pros

- Faster inference
- Less energy consumption
- Ideal for embedded applications

Solution: Hardened ViT

Idea: Implement the Vision Transformer processing in hardware

Pros

- Faster inference
- Less energy consumption
- Ideal for embedded applications

Cons

- Expensive to implement
- Niche market
- Adaptability+Flexibility

The Model

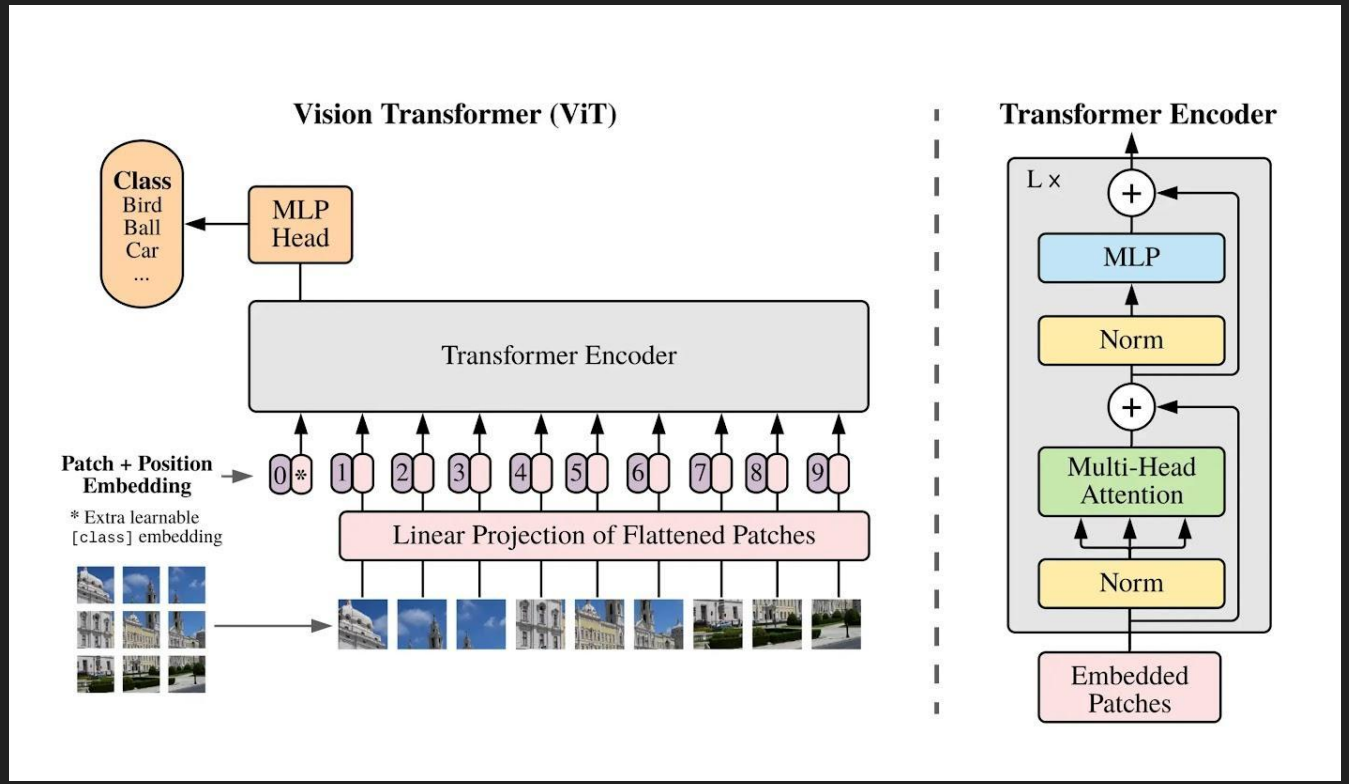
An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale

Alexey Dosovitskiy*,†, Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn*, Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby*,†

*equal technical contribution, †equal advising

Google Research, Brain Team

The Model

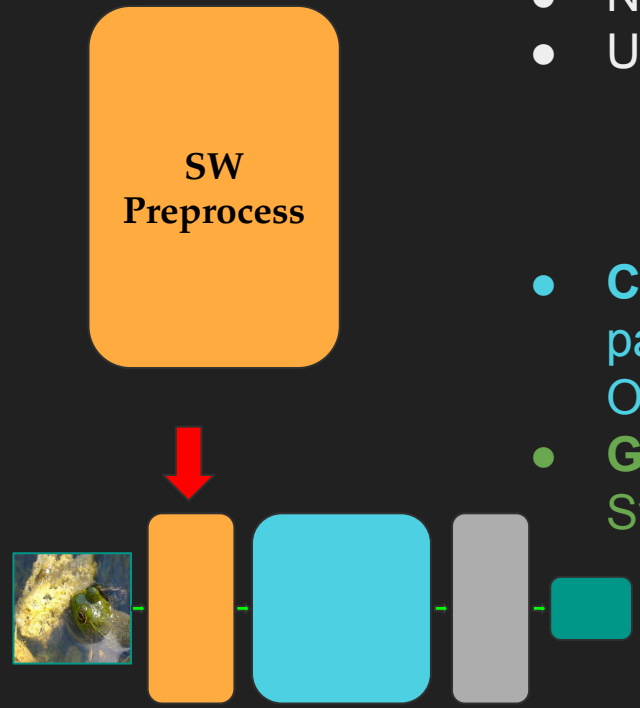

Vision Transformer (ViT)
Class
Bird
Ball
Car
...
MLP Head
Transformer Encoder
Patch + Position Embedding
* Extra learnable [class] embedding
0 * 1 2 3 4 5 6 7 8 9
Linear Projection of Flattened Patches
Transformer Encoder
L x
MLP
Norm
Multi-Head Attention
Norm
Embedded Patches

High-Level Design

→ SW Preprocess → Hardened ViT → SW Label Mapping → "Bullfrog"

Method: SW Preprocess

- Resize Image to 224 x 224
- Normalize all RGB values to float32 in range [0,1]
- Use OpenCV Library

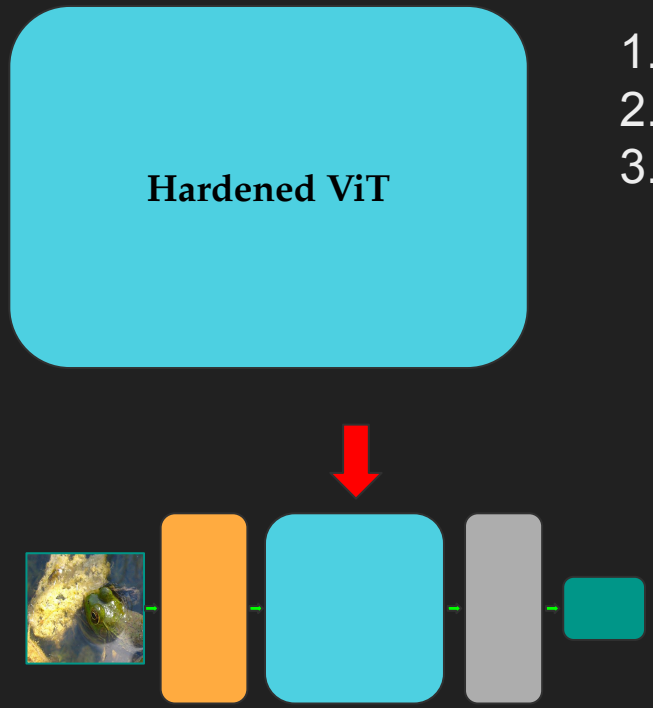


- **Current Implementation:** This is absorbed into the patch-embedding block, and gets put on the stack as a OpenCV Mat object
- **Goal:** Image gets written to the Hardened ViT via a SystemC FIFO

Method: SW Label Mapping

- Vision Transformer Block returns an integer
- Maps the integer to its corresponding label string

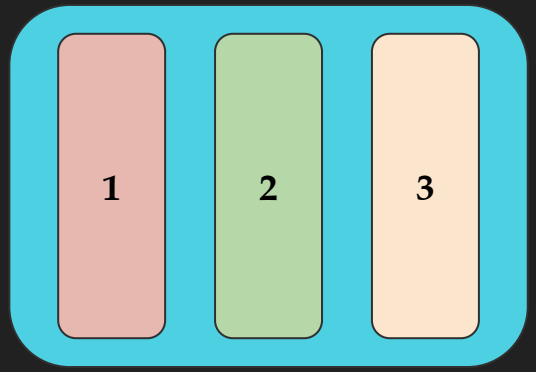
Hardened ViT



Broken up into three distinct modules:

1. Patch+Position Embedding
2. Transformer Encoder
3. Fully Connected (MLP)

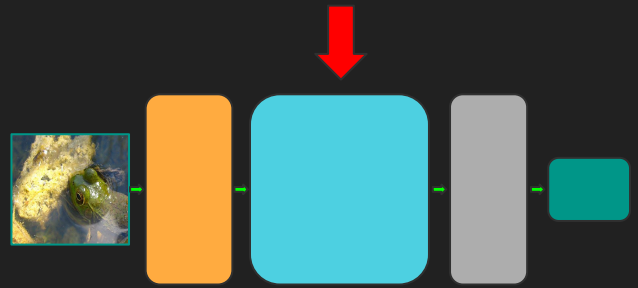
Hardened ViT



Broken up into three distinct modules:

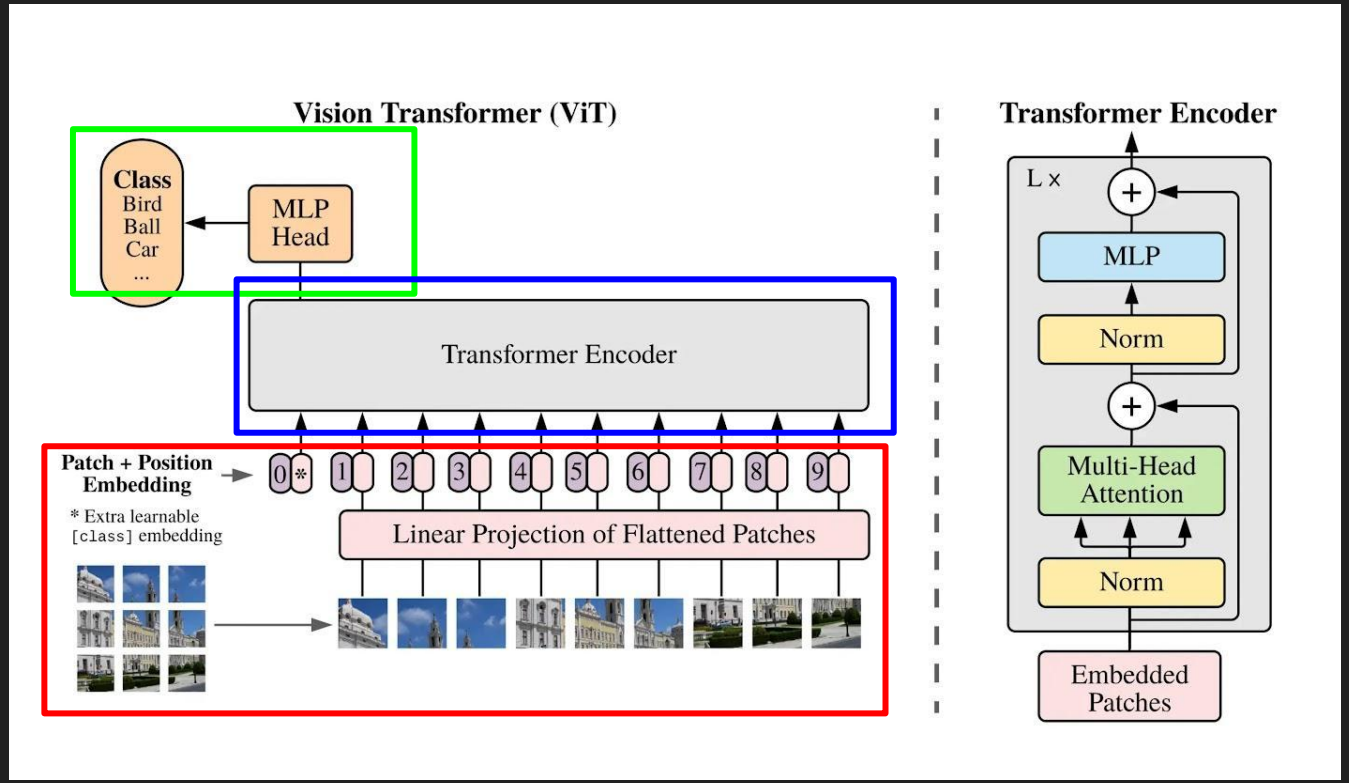
1. Patch+Position Embedding
2. Transformer Encoder
3. Fully Connected (MLP)

★ Each implemented in SystemC as an SC_MODULE



SYSTEMC™

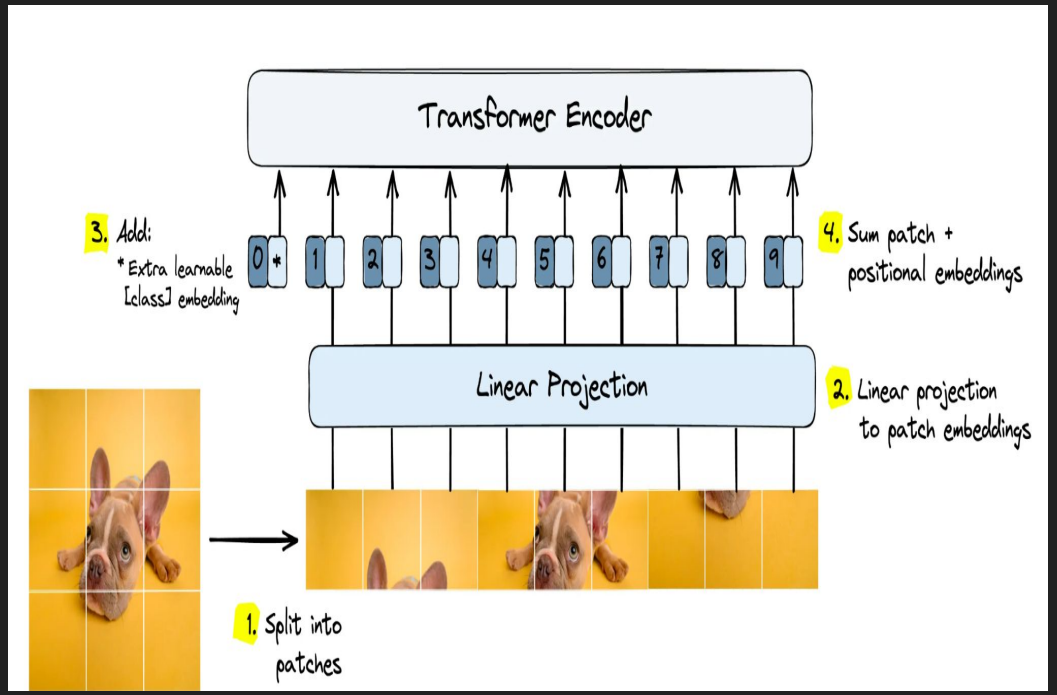
Hardened ViT



`SC_MODULE(VisionTransformer)`

- Top-level module, connects the Patch Embedding, Transformer Blocks, and MLP head together
- Responsible for handling dataflow and synchronization between the submodules
- Communication Design (Current)
  - Each module has a start and done signal implemented using `sc_signal<bool>`
  - Top-level module implements `run()`, an SC_THREAD that sets the start signal and waits for done signal
  - Input and output data is handled with c++ and is stored on the stack
- Communication Design (Future)
  - Migrate to more SystemC and less c++ — goal is to have most data moved via SystemC
  - Each module has a done signal, but start is sensitive on receiving all of its necessary data
  - Move implementation closer to reality!

1. Embeddings (Patch+Position+Class)



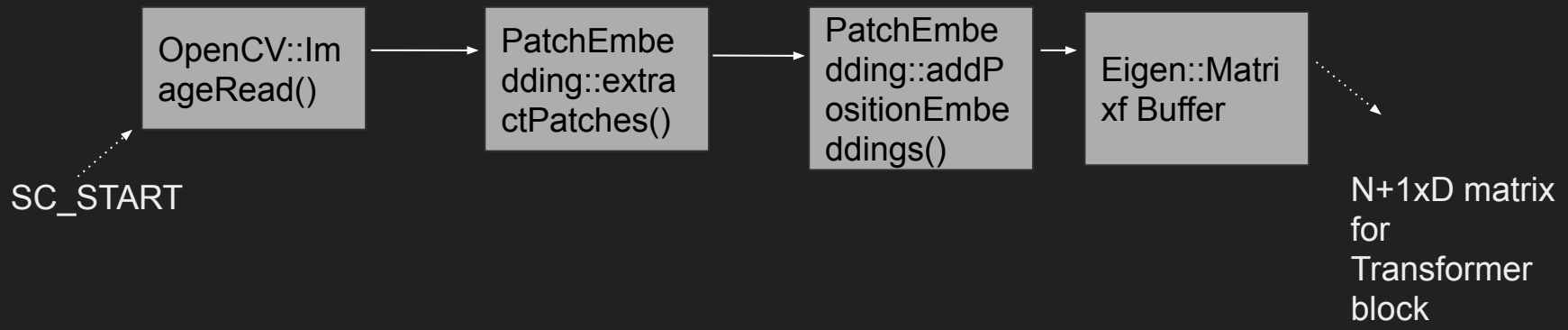
Why we need embeddings ?

- Similar to how text is processed in transformers (tokens).
- Unlike text (where token positions are explicit), patches have no inherent spatial order

How is it done ?

- Image patches + Position + Class embeddings (for classification)

1. Embeddings Design



- Image size: 224x224 x3
- No of patches = 224x224 x3 / 16x16 = 14x14 = 196 patches (N)
- Position Embeddings size: 196x768 (NXD)
- Final Embeddings size (+ 1 for class token): 197x768

224x224x3

16x16 patch size
196 patches (N)

196 X 768
(NXD)

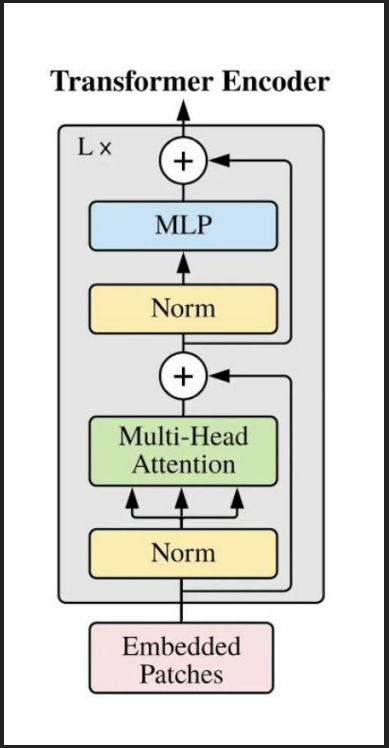
197x768
(N+1xD)

2. Transformer Encoder

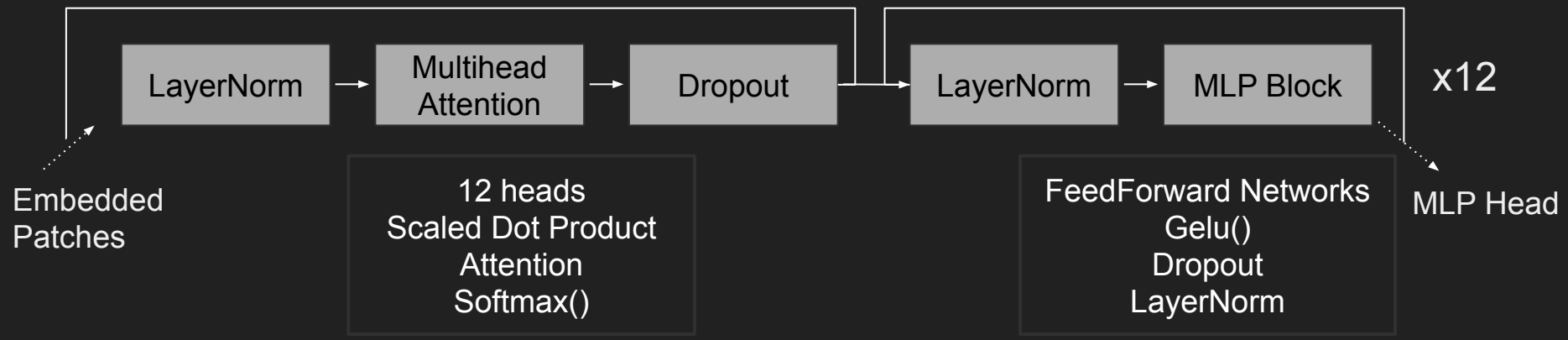
What is Transformer Encoder?

- A Transformer Encoder is like a smart **data processor**. It takes in chunks of information (like sentences or pieces of an image) and learns how these chunks are related to each other.

- We use it to find patterns and connections in data, making it useful for tasks like **understanding text** or recognizing parts of an image.



Transformer Encoder - Architecture



- **LayerNorm**: scale the data to standard normal distribution
- **MHA**: capture relationship between different patches
- **Dropout:**disable some scores to prevent overfitting
- **MLP block**: transform the features
- **Residual Connection**: output += input

224x224x3
16x16

(196+1)x768

197x(768/12)

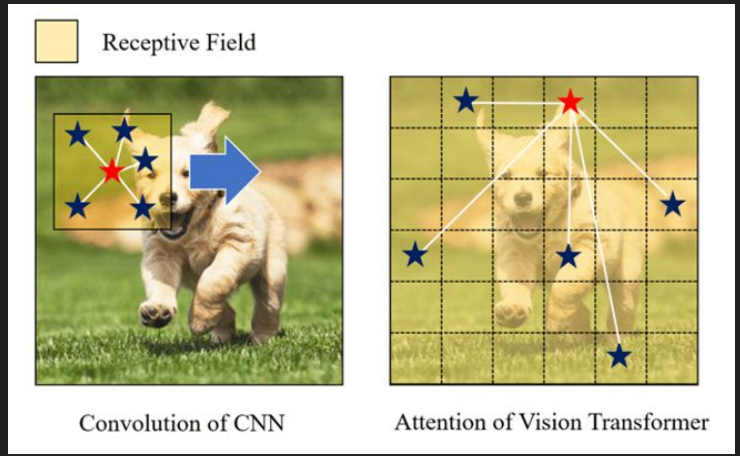
197x768

197x3072

197x768

Why we need Vision Transformer

While CNNs have been highly effective, they often require extensive computational resources and large datasets to achieve optimal performance. Additionally, CNNs primarily focus on local feature extraction, which can limit their ability to capture long-range dependencies within images. Vision Transformers, on the other hand, leverage self-attention mechanisms to model global relationships, potentially addressing these limitations.

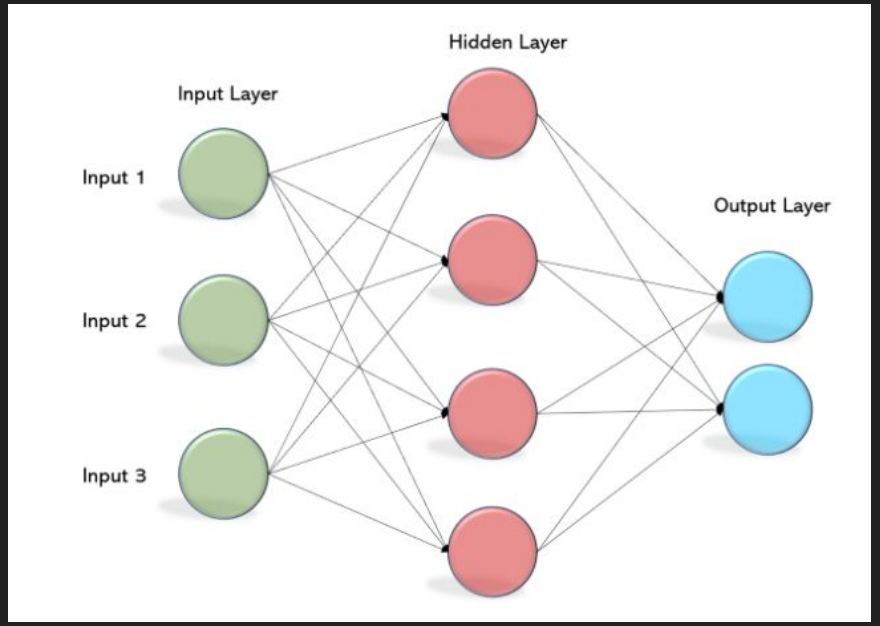


Source: https://medium.com/@mohankrishce/introduction-of-cnn-killer-part-i-43dec2e10811

What is Multi-Layer Perceptron (MLP) classifier ?

- **Definition**: The MLP Classifier refers to a Multi-Layer Perceptron (MLP), which is a type of neural network used for classification tasks. It is composed of multiple layers of neurons where each layer is fully connected to the next layer.

- **Applications**: This classifier is widely used in deep learning for various pattern recognition tasks including image recognition, speech recognition, and text classification, thanks to its ability to learn complex patterns in large datasets.



Source:
https://python.plainenglish.io/building-the-foundations-a-step-by-step-guide-to-implementing-multi-layer-perceptrons-in-python-51ebd9d7ecbe

Architecture of the MLP Classifier in System c

Input CLS Token

1x768

The token, a fixed-size vector output from the Transformer module.

First Linear Layer (W1)
z = W1 * x + b1

Operation: Multiplies the input vector by weight matrix W1 and adds bias b1.
Purpose: Expands the feature space to capture more complex patterns.

Weights 768 to 3078

1 * 3078

Activation Function (GELU)

Operation: Applies the Gaussian Error Linear Unit (GELU) activation to introduce non-linearity.
Purpose: Enhances the model's ability to learn complex relationships.

1 * 3078

Second Linear Layer (W2)
y = W2 * a + b2

Operation: Multiplies the activated features by weight matrix W2 and adds bias b2.
Purpose: Projects the features back to the number of desired classes

Output

1 * 1000

The final logits represent the raw, unnormalized scores for each class.

Testbench

Purpose:

- The testbench is like a lab where we test how well our Vision Transformer works.
- It makes sure the model is:
  - Correct: Does it classify images correctly?
  - Fast: How long does it take to give a result?

What does it do?

- Loads Pre-Trained Models: Reads model weights and configurations for testing.
- Processes Test Images: Feeds images to the model and gets predictions.
- Measures Performance: Calculates how fast and accurate the model is.

Testbench

1. Load Weights:
   - Reads model weights (pre-trained configurations) from files.
   - Applies the weights to key components like Patch Embedding, Encoder, and Classification Block.
2. Run Inference:
   - Feeds an input image (e.g., a picture of a fish) into the model.
   - The model processes the image and predicts a label (e.g., "fish").
3. Measure Performance:
   - Tracks the time taken for processing (speed).
   - Compares the output label with the expected label (accuracy).

State of Project + Results

- **Current Metric:** Evaluate block outputs against PyTorch model
- **Future:**
  - Evaluate model performance–compare classification accuracy with PyTorch
  - Evaluate processing speed with different forms of timing annotations

```
Output before entering transformer layers:
torch.Size([1, 197, 768])
tensor([[[-3.3752e-03,  1.3522e-02, -4.8854e-01,  ..., -5.6801e-03,
          -7.6900e-04, -7.5266e-01],
         [ 1.1185e-01, -2.1032e-01,  9.1745e-01,  ..., -3.4384e-02,
          -1.3114e-01, -8.4285e-02],
         [ 4.4953e-02, -1.0399e-02,  7.1369e-01,  ...,  6.9410e-02,
           8.3195e-03, -2.0526e-02],
         ...,
         [-6.0961e-02, -2.1622e-02,  5.9429e-02,  ...,  2.0960e-02,
           6.3235e-03, -2.4500e-01],
         [-1.1602e-01,  4.7043e-03,  1.5457e-01,  ...,  4.3442e-02,
           9.9756e-03, -3.4610e-01],
         [-5.8679e-02,  2.4110e-02,  4.3248e-01,  ..., -7.7987e-02,
          -1.8699e-01, -3.0705e-01]]], grad_fn=<SliceBackward0>)
```

Output from PyTorch hook before transformer layers

```
    [-3.3752e-03, 1.3522e-02, -4.8853e-01, 6.8525e-04, 7.5520e-01, 9.1354e-02, -2.5144e-0
2, 2.3088e-02, 6.3520e-02, -4.1793e-01, -1.3014e-02, -1.5707e-02, -1.6860e-02, -1.4696e-0
2, -2.4031e-02, -8.5413e-03, -1.4236e-03, 1.2781e-01, 6.0579e-02, -9.8877e-03, -1.0620e-0
1, -8.5113e-03, -1.0014e-02, -3.8575e-02, -1.7012e-03, 9.9492e-02, 4.7088e-03, -1.9241e-0
2, 7.7220e-02, -6.0349e-02, -1.3958e-03, 3.4471e-02, -3.1603e-02, 1.4979e-02, 4.3975e-02,
 3.7104e-03, 8.8529e-02, 6.6220e-03, -3.8641e-03, 1.5475e-02, 1.1901e-02, 1.5300e-02, 3.6
```

Output from SystemC Patch Embedding Block

Lessons Learned

- Start earlier 😔
- SystemC is a nice tool
- Working with a larger group brings up new advantages/disadvantages
  - Communication is very important
  - More people => more time spent on coordination (and merges)
  - More people => more precise task allotment and definition of done
  - More people => more opportunity for cool projects like this

Project Conclusions

- Implementing a Vision Transformer in hardware is doable!
- Weights are difficult to deal with
- SystemC is very nice compared to RTL, but dealing with c++ brings up its own difficulties when compared to Python — someone should make a SystemPython library

Thank You